

Student Grade Portal - Frontend Architecture Guide

This document outlines the architecture, layout, state flows, routing policies, and design patterns of the React-based frontend client.

1. Core Framework & Tech Stack

The frontend is a modern SPA (Single Page Application) configured with:

- **React:** Declarative component UI engine.
- **Vite:** Fast, lightweight compilation build tool and local dev proxy.
- **React Router DOM:** Client-side declarative routing and redirection logic.
- **Vanilla CSS:** Flexible, responsive styling system configured in `index.css` and `App.css`.

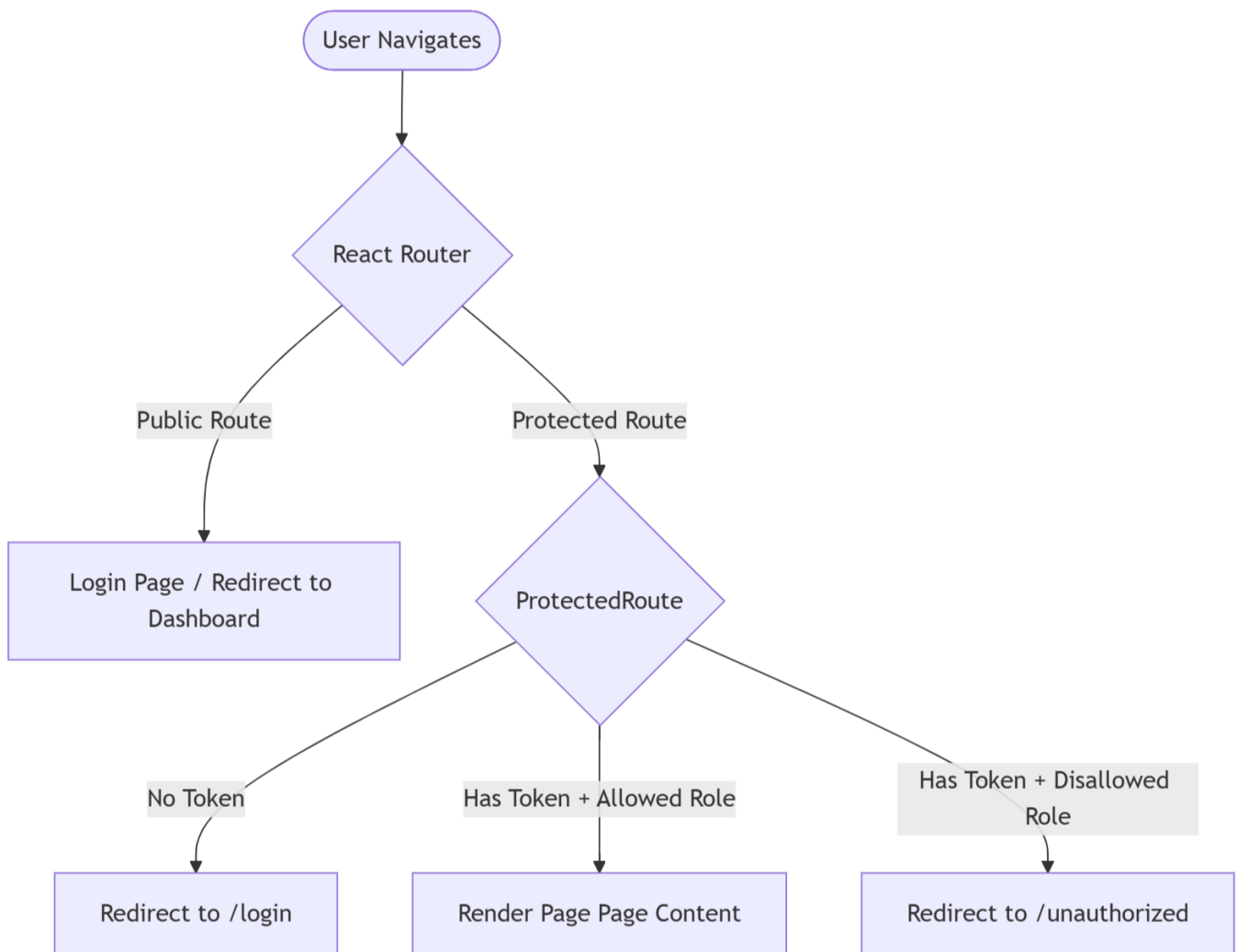
2. Frontend Project Directory

The React codebase is located inside the `/frontend/src` directory:

```
frontend/src/
├── App.css           # App-level styling variables
├── App.jsx           # Navigation route declarations & authorization guards
├── index.css         # Main stylesheet (color tokens, spacing, utilities)
├── main.jsx          # Application bootstrap mounting into index.html
├── assets/           # Image resources and vectors
├── components/       # Reusable child components
│   ├── CsvImporter.jsx # Multi-part CSV parsing and upload panel
│   ├── GradeChart.jsx  # Visual representation of scores
│   ├── Navbar.jsx      # Top application branding banner
│   └── Sidebar.jsx      # Contextual dashboard navigation panel
├── pages/            # Parent route containers
│   ├── ChangePassword.jsx # User security updates page
│   ├── Dashboard.jsx     # Home dashboard (stats, aggregations)
│   ├── GradeList.jsx     # Grading view (CRUD for grades)
│   ├── Login.jsx         # Credentials access screen
│   ├── StudentList.jsx   # Profiles folder (CRUD for students)
│   ├── Unauthorized.jsx  # Redirection fallback page
│   └── UserList.jsx       # Accounts console (Admin-only CRUD)
└── services/         #
    └── api.js           # Unified fetch interface with JWT token extraction
```

3. Route Access Guards & RBAC

Client-side security and Role-Based Access Control (RBAC) are declared directly inside `App.jsx`:



Protection Guards (Implementation detail)

- **ProtectedRoute** : Enforces authentication and checks authorization properties. If the route contains an `allowedRoles` restriction list and the logged-in user's role isn't matching, the request is redirected to `/unauthorized` .

```
function ProtectedRoute({ children, allowedRoles }) {
  const user = api.getCurrentUser();
  const token = localStorage.getItem('token');
  if (!token || !user) return <Navigate to="/login" replace />;
  if (allowedRoles && !allowedRoles.includes(user.role)) return <Navigate to="/unauthorized" replace />;
  return children;
}
```

- **PublicRoute** : Blocks authenticated users with active tokens from navigating back to the login screen, sending them directly to `/dashboard` .

4. Networking, State & Auth Lifecycle

All network tasks are isolated inside `api.js`:

Token Storage & Header Injection

- On login success, the client extracts the JWT token and writes it, along with username/role objects, to browser-managed

`localStorage` .

- Every subsequent fetch request calls `getHeaders()` , which automatically formats and injects the header if a token is present:

```
headers['Authorization'] = `Bearer ${token}`;
```

Interceptor Flow

To cleanly intercept and handle expiration constraints, calls run through `handleResponse()` :

- If the backend returns `401 Unauthorized` or `403 Forbidden` (expired or compromised token):
 - i. The client deletes the cached token from `localStorage` .
 - ii. User profiles are wiped.
 - iii. The client enforces window redirection to `/login` to prompt a fresh login session.



5. Shared Core Components

- **CsvImporter** : Interacts with `api.importCsv()` . Rather than loading simple forms, it leverages a drag-and-drop file uploader that formats a `MultipartFile` body, streams it to `/api/grades/import` , and prints raw logs returned by the backend service.
- **GradeChart** : Uses chart interfaces to map student grades across assignments, showing grade trends, class averages, and feedback charts.
- **Navbar & Sidebar** : Reads user object context dynamically to adapt the menu sidebar links (hiding Student lists from Students or Admin consoles from Teachers).